

1. Two Sum Easy

Acceptance: 56.9% | [LeetCode](#)

Given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to target*.

You may assume that each input would have **exactly one solution**, and you may not use the *same* element twice.

You can return the answer in any order.

Example 1:

```
**Input:** nums = [2,7,11,15], target = 9
**Output:** [0,1]
**Explanation:** Because nums[0] + nums[1] == 9, we
return [0, 1].
```

Example 2:

```
**Input:** nums = [3,2,4], target = 6
**Output:** [1,2]
```

Example 3:

```
**Input:** nums = [3,3], target = 6
**Output:** [0,1]
```

Constraints:

- $2 \leq \text{nums.length} \leq 104$
- $-109 \leq \text{nums}[i] \leq 109$
- $-109 \leq \text{target} \leq 109$
- **Only one valid answer exists.**

Follow-up: Can you come up with an algorithm that is less than $O(n^2)$ time complexity?

Claude Opus Solution

Two Sum - Complete Solution Guide

1. Intuition

The key insight is recognizing this as a **complement search problem**. For each number `x` in the array, we're looking for its complement `target - x`.

Instead of searching the entire array for each complement ($O(n^2)$), we can use a **hash map** to remember what numbers we've seen and their indices. This transforms the inner search from $O(n)$ to $O(1)$.

The clever trick: As we iterate, we store `target - num` (what we need) as the key, not `num` itself. When we encounter a number that exists as a key in our map, we've found our pair!

2. Approach (Step-by-Step Pseudocode)

1. Create an empty hash map (dictionary)
2. For each index `i` and number `num` in the array:
 - a. Check if `num` exists as a key in the hash map
 - If YES: We found our pair! Return `[hash_map[num], i]`
 - b. If NO: Store `(target - num) -> i` in the hash map
 - This records "if I see `(target - num)` later, it pairs with index `i`"
3. (Problem guarantees a solution exists, so we'll always return in step 2a)

3. Python Solution

```
class Solution:
    def twoSum(self, nums: list[int], target: int) -> list[int]:
        # Hash map to store: complement_needed -> index_that_needs_it
        # Key: the number we're looking for
        # Value: the index of the number that would pair with it
        seen = {}

        # Single pass through the array
        for i, num in enumerate(nums):
            # Check if current number is a complement someone was looking for
            if num in seen:
                # Found it! seen[num] is the index of the number that needs 'num'
                # i is the current index
                return [seen[num], i]

        # Current number wasn't found as a complement
        # Record what complement WE need (target - num)
```

```

        # If we see (target - num) later, it should pair with index i
        complement = target - num
        seen[complement] = i

    # Problem guarantees exactly one solution, so we'll never reach here
    return []

```

Walkthrough with Example

```

nums = [2, 7, 11, 15], target = 9

i=0, num=2:
- Is 2 in seen? No (seen is empty)
- Store complement: seen[9-2] = seen[7] = 0
- seen = {7: 0}

i=1, num=7:
- Is 7 in seen? YES! seen[7] = 0
- Return [0, 1] ✓

```

4. Complexity Analysis

Time Complexity: $O(n)$

- We iterate through the array exactly once
- Each hash map lookup and insertion is $O(1)$ average case
- Total: $O(n)$ operations

Space Complexity: $O(n)$

- In the worst case, we store $n-1$ entries in the hash map before finding the answer
- The hash map stores at most n key-value pairs

5. Alternative Approaches

Approach 1: Brute Force - $O(n^2)$ time, $O(1)$ space

```

def twoSum(self, nums: list[int], target: int) -> list[int]:
    # Check every pair
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] + nums[j] == target:
                return [i, j]

```

- Simple but slow for large inputs

Approach 2: Sort + Two Pointers - $O(n \log n)$ time, $O(n)$ space

```

def twoSum(self, nums: list[int], target: int) -> list[int]:
    # Store original indices before sorting
    indexed_nums = [(num, i) for i, num in enumerate(nums)]
    indexed_nums.sort() # Sort by value

    left, right = 0, len(nums) - 1
    while left < right:
        current_sum = indexed_nums[left][0] + indexed_nums[right][0]
        if current_sum == target:
            return [indexed_nums[left][1], indexed_nums[right][1]]
        elif current_sum < target:
            left += 1
        else:
            right -= 1

```

- Useful when you need to find multiple pairs or variations of the problem
- Sorting destroys original indices, so we need to track them

Summary Table

Approach	Time	Space	Notes
Brute Force	$O(n^2)$	$O(1)$	Simple, no extra space

Approach	Time	Space	Notes
Hash Map	$O(n)$	$O(n)$	Optimal for single pass
Sort + Two Pointers	$O(n \log n)$	$O(n)$	Good for variations

GPT-5 Solution

1) Intuition (Key Insight / Pattern)

This is a classic “find pair with given sum” problem.

For each number x , we need another number y such that:

- $x + y = \text{target} \iff y = \text{target} - x$ (the **complement**)

If we can quickly check whether the complement has appeared before, we can solve it in one pass. A **hash table (dictionary)** lets us do that in $\sim O(1)$ average time per lookup.

2) Approach (Step-by-step Pseudocode)

One-pass hash map (store seen numbers $\rightarrow$ their indices):

```
seen = empty map // value -> index

for i from 0 to n-1:
    x = nums[i]
    complement = target - x

    if complement in seen:
        return [seen[complement], i] // found the pair

    seen[x] = i // remember where we saw x

// problem guarantees exactly one solution, so we will return inside the loop
```

Key detail: insert x **after** checking, so you never use the same element twice.

3) Python Solution (Clean + Heavily Commented)

```
from typing import List

class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        # seen maps: number_value -> index_where_it_appeared
        # This allows O(1) average-time lookups for complements.
        seen = {}

        # Iterate once through the array
        for i, x in enumerate(nums):
            # The number we would need with x to reach target
            complement = target - x

            # If we've seen the complement before, we found the unique solution
            if complement in seen:
                # Return indices: (index of complement) and (current index i)
                return [seen[complement], i]

            # Otherwise, remember this number's index for future elements
            # We store AFTER the check to avoid using the same element twice.
            seen[x] = i

        # Given constraints say there is exactly one solution, so we should never reach here.
        raise ValueError("No two sum solution exists (contradicts problem constraints).")
```

4) Complexity Analysis

- **Time Complexity:** $O(n)$ average

We do one pass over `nums`, and each dictionary lookup/insert is **$O(1)$** on average.

- **Space Complexity:** $O(n)$

In the worst case, we store nearly every element in the hash map before finding the answer.

5) Alternative Approaches (Brief)

1. **Brute Force (nested loops)**

2. Check all pairs (i, j) with $i < j$

3. Time: $O(n^2)$, Space: $O(1)$

4. **Sort + Two Pointers (then map back to indices)**

5. Sort values with original indices, use `l/r` pointers to find sum

6. Time: $O(n \log n)$ due to sorting, Space: $O(n)$ (to keep indices)

7. More complex than the hash-map approach, but useful when you want ordered scanning patterns.